Olsker Cupcakes

Rapport

Udviklingsprojekt / Website prototype
Klasse A

Deltagere

Fabian Leonhard Strøm -
cph-fs180@stud.ek.dk - Fabian-str
Mikkel Fuglsig Nielsen -
mini0011@stud.ek.dk - MikkelMattern
Nali Ahmadi -
cph-na282@stud.ek.dk - Nali1234-lab

Dato: 17-04-2026

Indholdsfortegnelse

Indledning 3
Baggrund 3
Teknologivalg 4
Krav 5
Firmaets vision 5
User stories 5
Aktivitetsdiagram 6
As-Is Aktivitetsdiagram - Olsker Cupcakes 6
To-Be Aktivitetsdiagram - Olsker Cupcakes 7
Domæne model og ER diagram 8
Domæne model 8
ER diagram 9
Navigationsdiagram 10
Særlige forhold 11
Status på implementation 12
Implementerede funktioner 12
Kendte mangler 13
Proces 14
Teamets planer for projektforløb 14
Planer for database 14
Planer for Java og Javascript 15
Planer for HTML og CSS 16
Projektforløbet i praksis 17
Forløbet for databasen 17
Forløbet for Java og Javascript 18
Forløbet for HTML og CSS 18
positive og negative oplevelser under projektforløbet 19
Positive oplevelser 19
1. Online samarbejde og kommunikation 19
2. Daglige møder og arbejdsstruktur 19
3. Forståelse, fleksibilitet og arbejdsmiljø 20
4. Feedbackkultur og løbende udvikling 20
Negative oplevelser 21
1. Struktur Udfordringer ved online samarbejde 21
2. Tidsbegrænsning og nødvendige kompromiser 21
Teamets erfaringer under projektforløbet, og tanker om forbedringer 22

Indledning

Formålet med denne rapport er at dokumentere udviklingen af et softwareprojekt rettet mod en specifik målgruppe. Målgruppen for rapporten er datamatikerstuderende på 2. semester eller tilsvarende niveau, som har den nødvendige tekniske forståelse til at følge de valg og overvejelser, der præsenteres.

Projektet omhandler udviklingen af en hjemmeside for en kunde, der ønsker at sælge sine produkter online. Kunden har en række overordnede krav til både funktionalitet og indhold, som danner grundlag for systemets design og implementering.

Under projektforløbet har gruppemedlemmerne arbejdet med forskellige roller inspireret af praksis i softwareudvikling. Disse roller omfatter blandt andet Scrum Master, Product Owner og Tech Lead, som hver især har haft ansvar for forskellige dele af projektets fremdrift og beslutninger.

Baggrund

Kunden er et bageri på Bornholm ved navn Olsker Cupcakes, som specialiserer sig i cupcakes. Virksomheden ønsker at få udviklet en hjemmeside, der gør det muligt for kunder at placere ordrer af cupcakes online, hvorefter det er muligt for kunden at afhente sine ordrer senere, når den er klar til afhentning.

Som udgangspunkt for opgaven er der udarbejdet en mockup af en mulig forside samt en række overordnede krav til systemets funktionalitet. Disse krav omfatter blandt andet muligheden for at sammensætte cupcakes ved valg af bund og top, samt at kunne afgive og håndtere ordrer.

Det er en del af opgaven at videreudvikle disse krav, afklare manglende funktionalitet og komme med forslag til løsninger.

Teknologivalg

- IntelliJ 2025.2.4
- Javalin: 6.7.0
- Javalin Rendering: 6.5.0
- Thymeleaf: 3.1.3.RELEASE
- Thymeleaf Extras Java 8 Time: 3.0.4.RELEASE
- SLF4J Simple: 2.0.17
- Jackson Databind: 2.18.3
- HikariCP: 6.2.1
- PostgreSQL JDBC Driver: 42.7.5
- JUnit Jupiter API: 5.12.0
- JUnit Jupiter Engine: 5.12.0
- CSS3
- pgAdmin4
- Git 2.53.0.windows.2
- Github Desktop
- PlantUML 1.2026.2
- Maven 4.0.0
- Hamcrest: 3.0
- Maven Compiler Plugin: 3.10.1
- Maven Shade Plugin: 3.4.1
- Packaging: jar
- PostgreSQL: bruges via JDBC-driver version 42.7.5
- Connection pool: HikariCP 6.2.1
- Jackson Databind: 2.18.3
- Java 25
- Javascript ECMAScript 6+
- HTML5
- Firefox 149.0.2 (64-bit)
- Google Chrome 147.0.7727.101

Krav

Firmaets vision

Olsker Cupcakes har som primært mål at sikre den bedst mulige kundeoplevelse. Som led i dette ønsker virksomheden at udvikle en hjemmeside, der gør det nemmere og hurtigere for kunderne at bestille deres produkter.

Formålet er at minimere ventetiden og dermed skabe en mere effektiv og brugervenlig bestillingsproces, hvilket afspejler virksomhedens fokus på og respekt for kundernes tid.

User stories

1. Som kunder kan jeg bestille og betale cupcakes med en valgfri bund og top, sådan at jeg senere kan køre forbi butikken i Olsker og hente min ordre.

2. Som kunde kan jeg oprette en konto/profil for at kunne betale og gemme en ordre.

3. Som administrator kan jeg indsætte beløb på en kundes konto direkte i Postgres, så en kunde kan betale for sine ordrer.

4. Som kunde kan jeg se mine valgte ordrelinjer i en indkøbskurv, så jeg kan se den samlede pris.

5. Som kunde eller administrator kan jeg logge på systemet med e-mail og kodeord. Når jeg er logget på, skal jeg kunne se min email på hver side (evt. i topmenuen, som vist på mockup'en).

6. Som administrator kan jeg se alle ordrer i systemet, så jeg kan se hvad der er blevet bestilt.

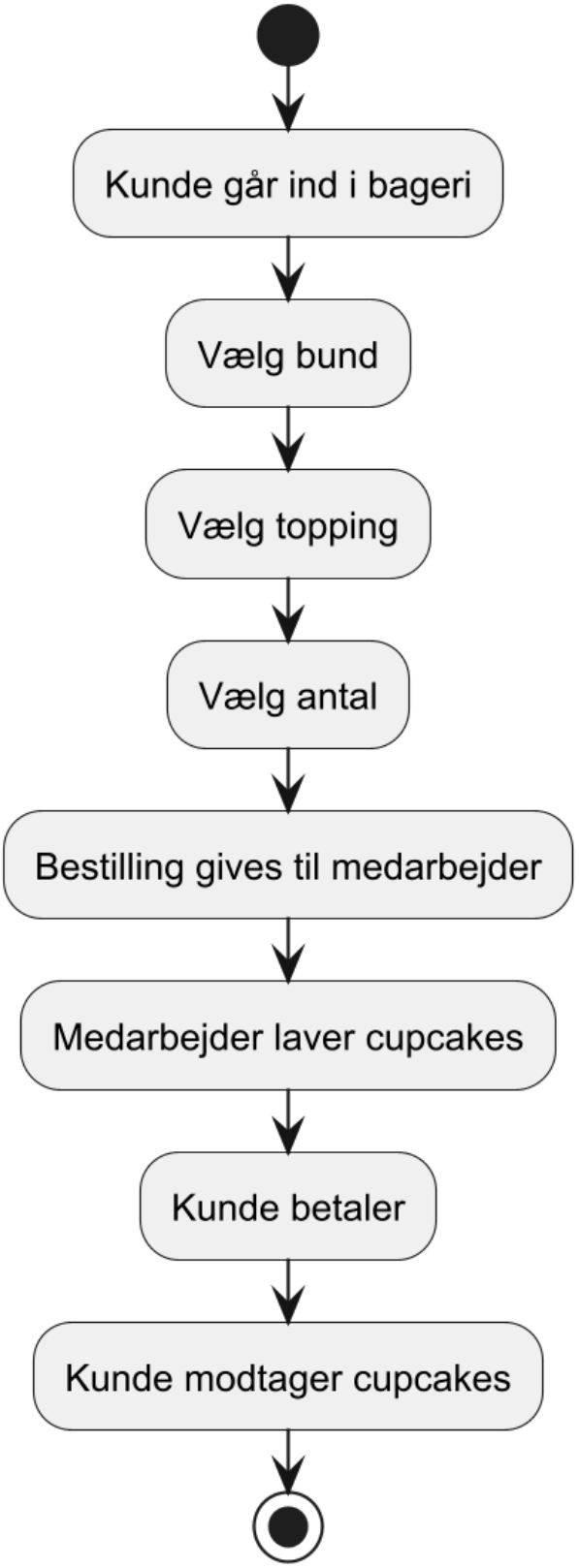
7. Som administrator kan jeg se alle kunder i systemet og deres ordrer, sådan at jeg kan følge op på ordrer og holde styr på mine kunder.

8. Som kunde kan jeg fjerne en ordrelinje fra min indkøbskurv, så jeg kan justere min ordre.

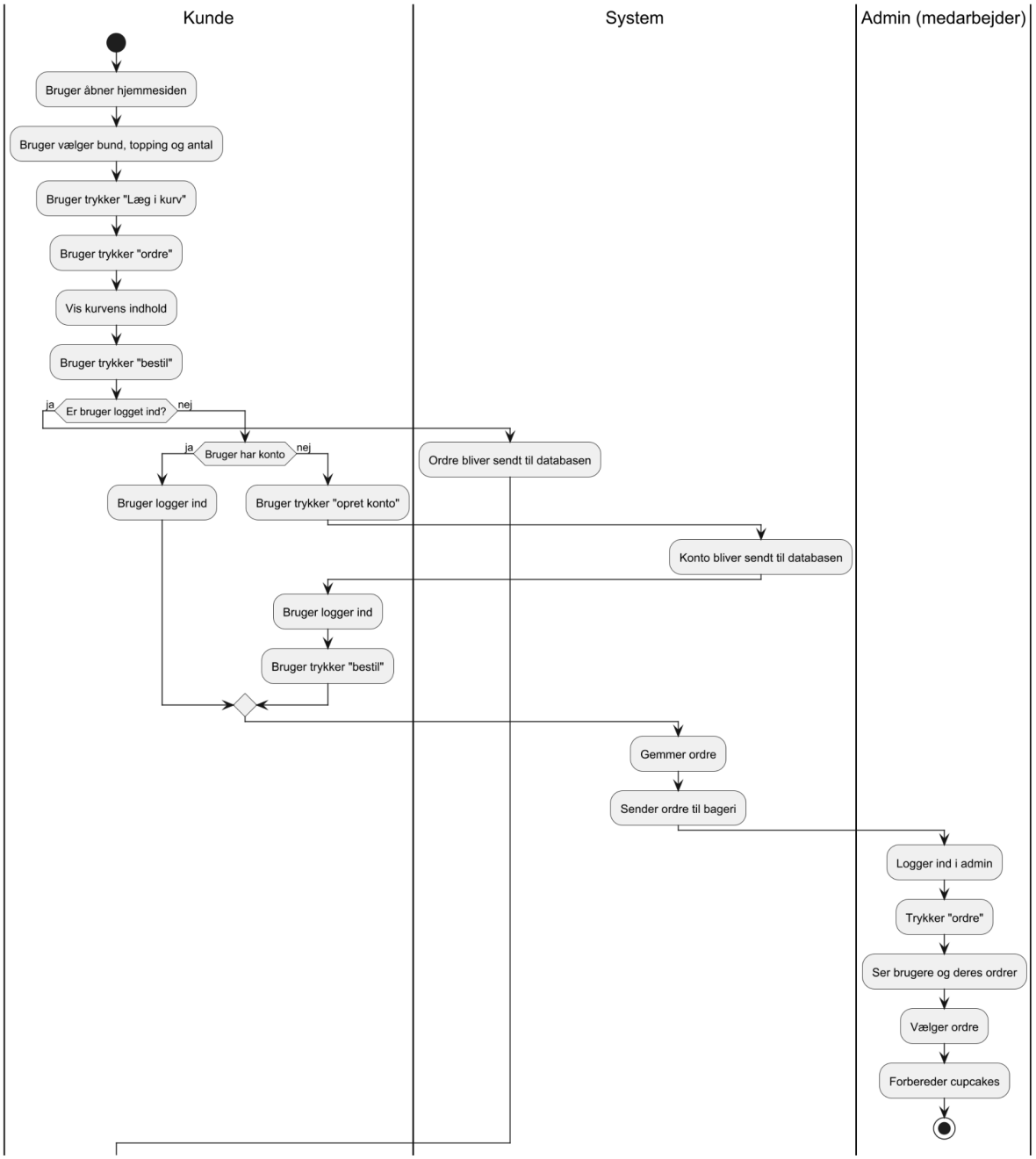
9. Som administrator kan jeg fjerne en ordre, så systemet ikke kommer til at indeholde ugyldige ordrer. F.eks. hvis kunden aldrig har betalt.

Aktivitetsdiagram

As-Is Aktivitetsdiagram - Olsker Cupcakes

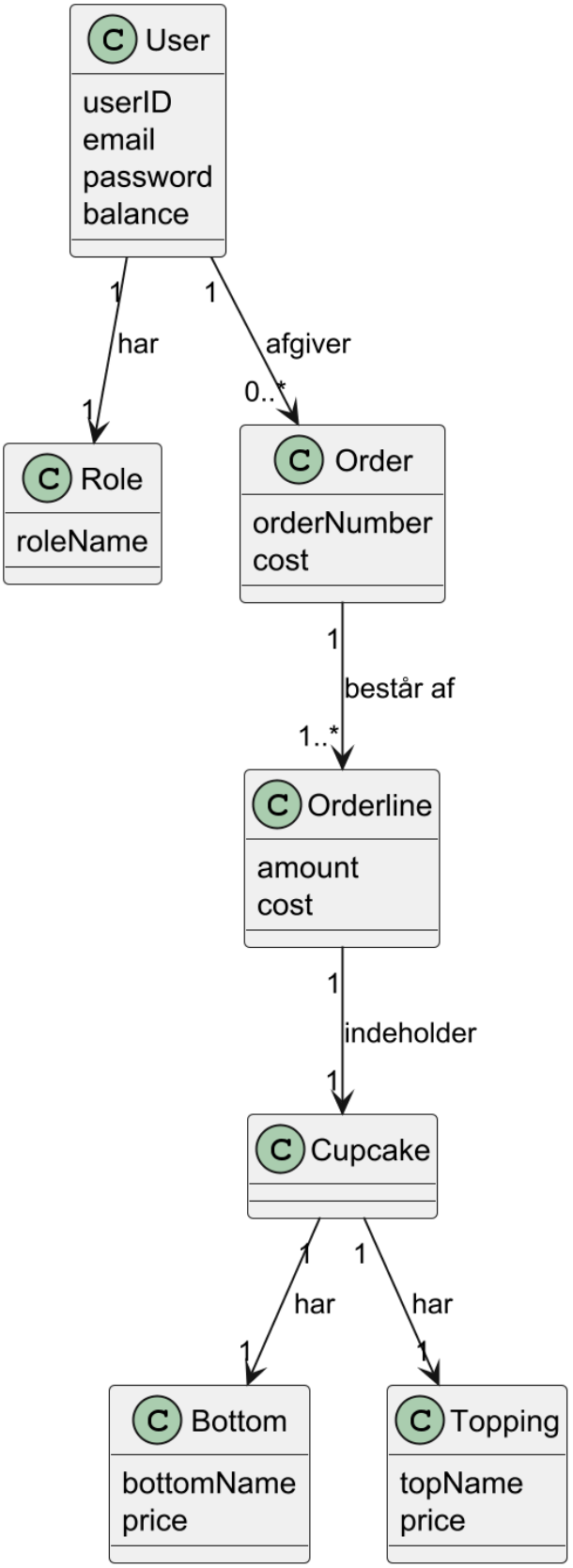


To-Be Aktivitetsdiagram - Olsker Cupcakes

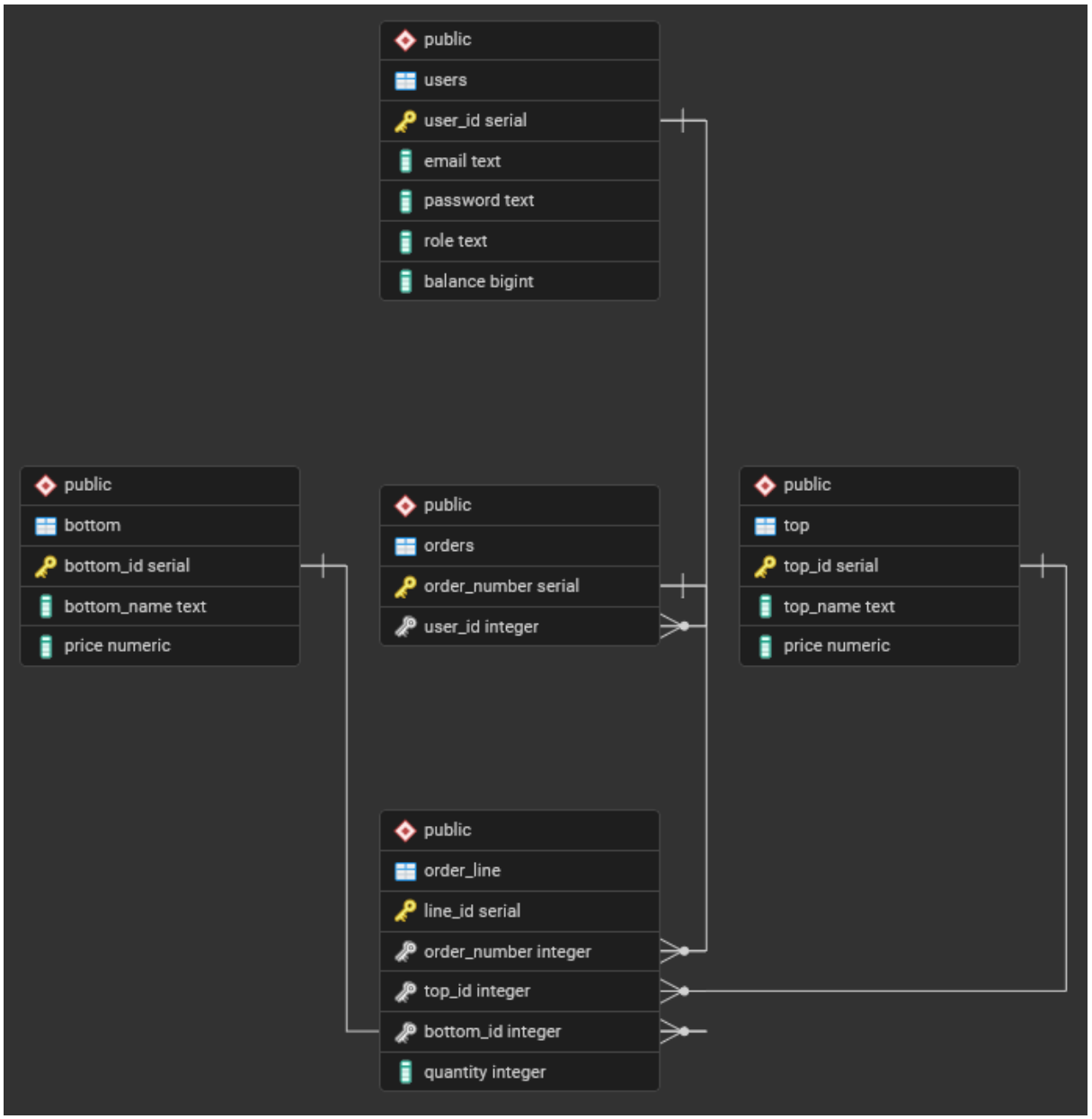


Domæne model og ER diagram

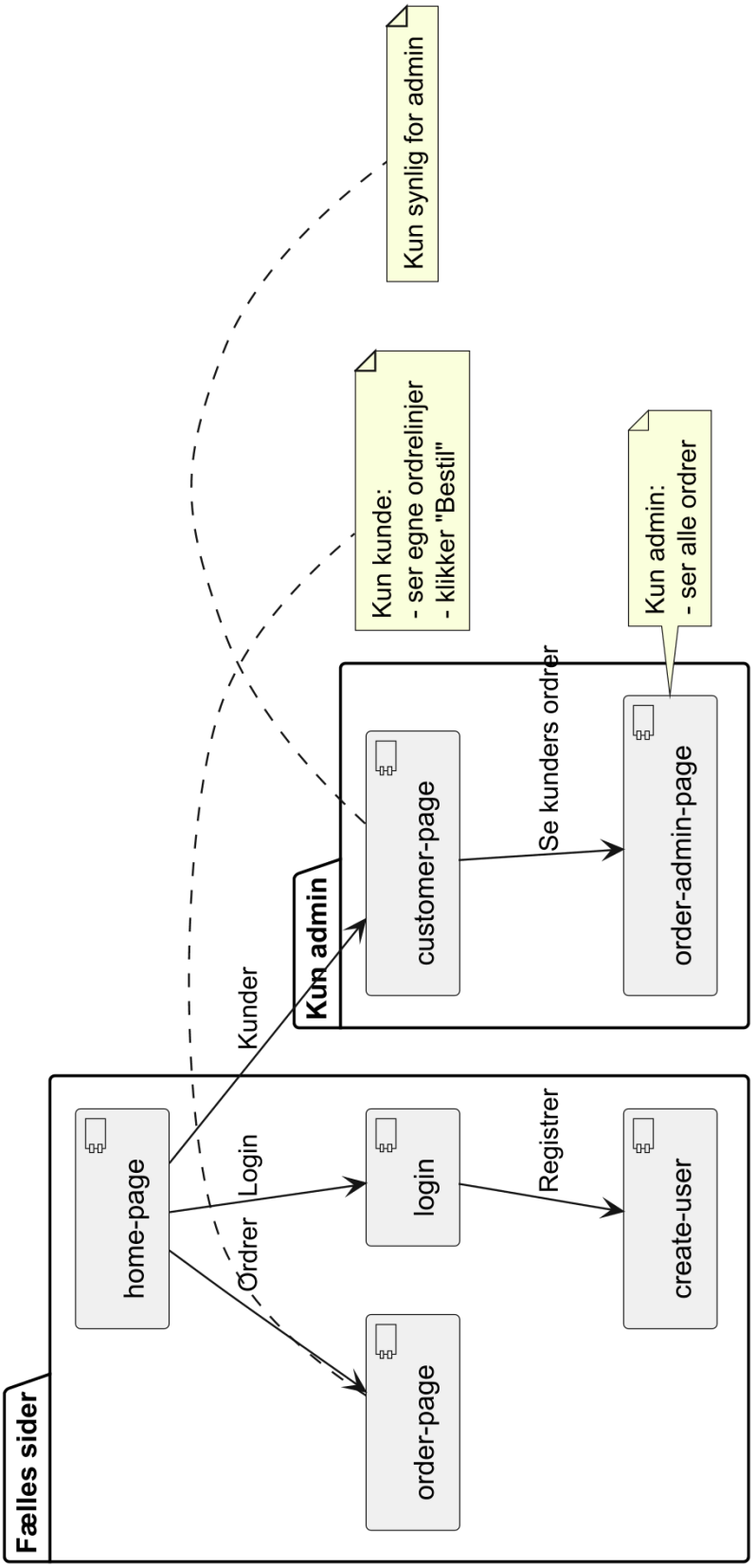
Domæne model



ER diagram


public
users
user_id serial
email text
password text
role text
balance bigint
public
bottom
bottom_id serial
bottom_name text
price numeric
public
orders
order_number serial
user_id integer
public
top
top_id serial
top_name text
price numeric
public
order_line
line_id serial
order_number integer
top_id integer
bottom_id integer
quantity integer

Navigationsdiagram



Særlige forhold

I projektet er der arbejdet med sessionsstyring for at håndtere information om den bruger, der er logget ind. Når en bruger logger ind, gemmes brugerens oplysninger i sessionen, hvilket gør det muligt at tilgå disse på tværs af sider. Dette bliver blandt andet brugt i frontend, hvor brugerens e-mail vises, samt til at styre hvilke funktioner der er tilgængelige.

I forhold til håndtering af fejl har vi valgt at anvende en selvdefineret DatabaseException. Denne bliver brugt i persistence-laget til at håndtere fejl i forbindelse med databasen. På den måde undgår vi at arbejde direkte med SQLExceptions i resten af applikationen, og det bliver lettere at forstå og håndtere fejl.

Validering af brugerinput er primært håndteret i forbindelse med login. Her bliver de indtastede oplysninger sammenlignet med data i databasen, og hvis oplysningerne ikke stemmer overens, gives der en fejl. Vi har i denne løsning valgt en simpel tilgang, hvor valideringen sker på serversiden.

I forhold til sikkerhed i login processen har vi arbejdet med en sessionsbaseret løsning. Når brugeren er logget ind, bruges sessionen til at afgøre, hvilke sider og funktioner brugeren har adgang til. Derudover har vi implementeret en opdeling i brugerroller, hvor kun brugere med rollen ADMIN har adgang til bestemte funktioner, såsom visning af kunder og ordrer. Passwords håndteres i denne løsning som klartekst, hvilket fungerer til formålet, men i en virkelig løsning bør dette håndteres mere sikkert, eksempelvis ved hashing.

I databasen har vi valgt at arbejde med brugerroller via et felt kaldet role. Denne værdi bliver i Java mappet til en enum (Role), hvilket gør det nemmere at arbejde med roller i koden. Rollen bliver efterfølgende brugt både i backend og i Thymeleaf til at styre, hvad brugeren må se og gøre.

Derudover har vi arbejdet med et separat test-schema i databasen. Dette gør det muligt at køre tests uden at påvirke den normale data. I forbindelse med tests bliver databasen nulstillet, så vi sikrer, at hver test starter fra samme udgangspunkt. Dette har gjort det lettere at teste funktionalitet i persistence-laget.

Status på implementation

Implementerede funktioner

- Kunder kan oprette profil og logge ind.
- Kunder kan bestille cupcakes med valgfri top og bund.
- Administrator kan sætte beløb ind på kunders profil direkte i Postgres.
- Kunder kan se de tilføjede ordrelinjer i sin kurv.
- Kunder kan se prisen på hver ordrelinje i kurven.
- Kunder kan slette ordrelinjer fra deres kurv.
- Kunder kan se den totale pris på produkterne i kurven.
- Kunder kan bestille ordren i deres kurv.
- Kunder og administrator kan se deres email på hver side når de er logget ind.
- Administrator kan se alle ordrer der er blevet bestilt.
- Administrator kan slette ordrer i systemet.
- Administrator kan se alle kunder i systemet.
- Responsivt design.
- Password-bekræftelse når man opretter sig som bruger.

Kendte mangler

Selvom disse mangler ikke er direkte krav, føler vi stadig at de ville have forbedret produktet og været vigtige at have med hvis vi havde mere tid til at implementere dem.

- Manglende validering af email input når man opretter sig.
- Manglende kryptering af password i Postgres.
- Ingen funktion for administrator til at tilføje penge til brugernes konti direkte fra hjemmesiden.
- Ingen funktion til at ændre brugernes rolle.
- Ingen evne til at yderligere redigere kurven, som at ændre valgte top, bund eller antal.
- Ingen grænse på hvor mange gange man kan forsøge at logge ind.
- Ingen mulighed for at nulstille password.
- Ingen fejlbesked hvis man bestiller en tom ordre.

Proces

Teamets planer for projektforløb

Planer for database

For at lave vores database har vi lavet en analyse af user stories for at finde kundernes behov. Vi har identificeret via US2 at vi skulle holde styr på brugere, som vi gør med en “users” tabel, så de kan logge ind. “users” tabellen har en primærnøgle som bliver kaldt “user_id”, en “email” og “password” så brugeren kan oprette bruger med deres email og et password, som beskrevet i US5. Der bliver også beskrevet i denne user story at man skal kunne logge ind som administrator eller bruger, og for at holde styr på disse roller, har vi oprettet en “role” attribut, denne er ved default “USER”. I US3 er der et krav om at man som administrator skal kunne tilføje et beløb til brugerens saldo, og af den grund har vi oprettet et “balance” felt. Vi har også haft overvejelse om hvorvidt vi skulle have et adressefelt med, i tilfælde af at Olsker Cupcakes tilbyder udlevering, men det besluttede vi os for ikke at inkludere, da vi ikke vidste om de tilbyder denne service.

Vi har på baggrund af US1 identificeret behovet for at gemme en ordre i databasen, dette gør vi med en "orders" -tabel. Ordre Tabellen har kun to attributter, den ene “order_number” som er dens primærnøgle, og en “user_id” som er en fremmednøgle forbundet med “users” primære nøgle.

Vi har tilføjet henholdsvis en “top” og “bottom” tabel for at holde styr på den valgte bund og top i en ordre, dette har vi gjort på baggrund af US1. I disse tabeller har vi fået data fra selve opgave som skulle tilføjes, her har vi fået pris og navn på henholdsvis toppe og bunde, hvor vi så har oprettet en “name”, en “price” i begge samt et ID som fungerer som den primære nøgle til dem. Vi overvejede også at tilføje en beskrivelse til tabellerne for toppe og bunde, men besluttede at lade være, da vi ikke har fået data om hvad de forskellige toppings og bunde indeholder, og derfor ville skulle gætte på beskrivelserne.

Da en ordre kan have mange cupcake dele, og cupcake dele kan være en del af flere ordre, så har vi oprettet en “order_line” tabel, som består af et “line_id”, som er den primære nøgle, et “order_number” som er en fremmednøgle der hænger sammen med “orders” primære nøgle, en “top_id” og en “bottom_id” som også er fremmednøgler og hænger sammen med de respektives primære nøgler, og til sidst en “quantity”. Vi havde også overvejet at lave en “category” tabel, som ville sætte hver cupcake del i en kategori, som sød, som chocolate, eller savory, som blue cheese, men som nævnt før ved vi ikke tilstrækkelig meget om produktet for at kunne tildele delene en kategori, og derfor valgte vi at forkaste den tabel.

For at sikre at vores database fungerer korrekt, har vi valgt at oprette et separat testskema kaldet 'test' i samme database som produktionsdatabasen. Dette gør, at vi kan køre JUnit tests mod testskemaet uden at påvirke den rigtige data i 'public' schemaet. Vi valgte denne løsning frem for en helt separat testdatabase, da det er nemmere at vedligeholde, da begge schemas deler samme struktur.

Planer for Java og Javascript

Vi har valgt at bruge Model-View-Controller mønsteret fordi det adskiller vores applikation i tre forskellige lag, hvilket gør vores kode nemmere at udvide og vedligeholde. Hvis vi for eksempel ønsker at ændre hvordan data præsenteres, behøver vi kun rette i vores HTML templates, og hvis vi ønsker at rette i databaselogikken, behøver vi kun at rette i vores mappers, uden det påvirker de andre lag.

Vores overvejelser omkring hvordan vi skulle håndtere kurven inkluderede både at håndtere den i session scopet, og direkte mappe ordren ned i databasen, og så tjekke om hvorvidt den var blevet betalt, som man ville kunne gøre når man tjekkede ud. Vi valgte at gemme kurven i session scopet, da vi vurderede at vi kunne opnå en bedre UX, da alternativet ville kræve at man loggede ind før man tilføjede ting til kurven. Derudover inkluderede vores overvejelser også, hvorvidt data forbruget ville blive for stort hvis vi gemte enhver ordre der ikke var bestilt i databasen.

Vi har valgt at bruge Javalin som web framework til at håndtere vores HTTP requests og routing og Thymeleaf som template engine til at generere dynamisk HTML server-side, da det er disse teknologier vi er blevet eksponeret for og undervist i på uddannelsen, det er derfor forventet at vi bruger disse i vores opgave for at demonstrere vores evner til at arbejde med dem. Vi har derfor ikke overvejet andre web frameworks eller template engines, da vi er af den opfattelse at denne opgave har til formål at arbejde med dem.

Vi har bevidst valgt at holde Javascript på et minimum, da vi er blevet enige om at det er bedst at vores forretningslogik foregår i backend af koden. Dette gør vi af flere årsager, først og fremmest fordi det giver en øget sikkerhed, da det vil være vanskeligt for uautoriserede brugere og angribere at manipulere logikken, da de ikke vil være i stand til at se koden. Derudover vil det også være mere hensigtsmæssigt at teste vores kode, da det eneste test framework vi er bekendt med er JUnit, som er lavet specifikt til Java. Til sidst gør vi det fordi Java er et stærkt typet sprog, hvilket reducerer risikoen for fejl i forhold til JavaScript som er dynamisk typet. Dette gør Java mere forudsigeligt, da type-fejl opdages inden koden kører, frem for når den kører, som i Javascript.

Planer for HTML og CSS

Som nævnt i forrige afsnit valgte vi at bruge Thymeleaf som vores template engine, da det er den vi har fået undervisning omkring. Vi overvejede at bruge frontend frameworket Bootstrap, da nogle af holdets medlemmer har erfaring med det fra tidligere projekter. Fordelene ved et CSS framework som Bootstrap ville være at vi kunne spare tid på at skrive CSS, og stadig få et design der ville se professionelt ud. Vi valgte dog at skrive vores CSS fra bunden, da vi er blevet givet en mockup som vi skulle følge, og vurderede derfor at den øgede frihed ved selv at skrive vores CSS ville være nødvendig for at opnå et produkt så tæt på det ønskede resultat som muligt. Vores overvejelser omkring hvorvidt vi ville bruge andre teknologier har været begrænsede, da vi vurderede at tiden vi skulle bruge på at sætte os ind i dem ville opveje alle fordele vi eventuelt kunne få ved at bruge dem i dette projekt, samt at vi har vurderet at vi godt kunne opnå de tekniske krav uden brug af yderligere teknologier.

Da vi kun fik udleveret et mockup af forsiden, valgte vi at style vores andre sider efter designet på den. Vi talte om, hvorvidt vi kunne tænke os at designe login- og brugersiden på en anden måde, samt om de skulle være sider for sig selv, eller om de skulle designes som modals. Vi besluttede os for at lave dem som html sider for sig selv, da vi i det tilfælde ville kunne genbruge vores stylesheet i stedet for at style yderligere, og da routing ville være i overensstemmelse med hvad vi er blevet undervist i og har erfaring med. Vi valgte kun at lave en side for både administratorer og brugere, men differentiere dem ved, at man som administrator har adgang til yderligere to links i menuen, som er siderne som viser henholdsvis brugerne og ordrerne i databasen.

Valget omkring at have forskelligt indhold på samme side kom på baggrund af user story US5, denne user story beskriver at man skal kunne logge ind som både administrator og bruger, og vi valgte derfor at lave det på samme side for at reducere antallet af HTML sider vi skulle lave.

Vi har også på baggrund af de givne user stories analyseret de andre sider vi har skulle have med, som US1 som beskriver funktionerne af home-page.html, hvor man skal kunne vælge top, bund og antal, og derefter tilføje sin ordre til kurven. US2 beskriver funktionerne af login.html og create-user.html, hvor man henholdsvis skal kunne logge ind og oprette sig som bruger med email og password. US4 beskriver funktionen af order-page.html, hvor man som bruger kan se sine ordrelinjer i kurven, som for hver ordrelinje viser top, bund, pris og antal, og i bunden viser den totale pris for alle ordrelinjerne. US6 beskriver funktionen af order-admin-page.html, hvor man som administrator kan se kundernes bestilte ordrer, som viser brugerens email, ordrenummeret og den totale pris på ordren, og US7 som beskriver funktionen af

customer-page.html, hvor man som administrator har et overblik over de oprettede brugere, hvor der står deres navn, id, rolle og balance.

Som beskrevet i de tekniske krav valgte vi også at lave responsivt design, så hjemmesiden både kan bruges på større og mindre skærme.

Projektforløbet i praksis

Forløbet for databasen

Vi løb ind i en række problemer ved implementeringen af vores database, vi glemte blandt andet at kalde .next() på ResultSet i flere af vores metoder, før vi aflæste ResultSet, hvor vi skulle hente data fra databasen hvilket kastede en PSQLException med fejlbeskeden “ResultSet not positioned properly, perhaps you need to call next”. Denne exception bliver ikke fanget af og håndteret i vores kode, og på grund af det sendte Javalin en fejl 500 Server Error tilbage, da den ikke vidste hvad den skulle gøre med den ukontrollerede fejl. Vi løb også ind i problemer da vi satte vores testmiljø op, da vi bare havde kopieret den SQL som var blevet genereret af pgAdmin, og smidt den ind i vores “test” skema. Her var problemet at det ikke betyder noget, hvor man skriver SQL forespørgsel, men derimod hvad der står i det. Dette fiksede vi ved at ændre “public” til “test” i “CREATE SCHEMA public;”. For at oprette tabellerne korrekt tilføjede vi også skema præfikset til hver linje der stod for at oprette tabellerne.

Da vores ConnectionPool klasse bruger singleton mønsteret stødte vi også på problemet at når vi ville oprette forbindelse til vores test skema, fandtes der allerede en instans af ConnectionPool klassen, så da vi kaldte getInstance() i testmiljøet fik vi ikke en instans af test skemaet, men af public skemaet. Dette løste vi ved at tilføje endnu en metode til vores ConnectionPool klasse, som overskrev den eksisterende singleton instans med en ny instans med forbindelse til test skemaet.

Den sidste fejl vi løb ind i var da vi skulle lave setup() metoden som skal køre før alle test, der skal stå for at droppe “users”, “orders” og “order_line” tabellerne, samt alt deres data. Den skulle også stå for at tilføje en bruger og en ordre derefter. Fejlen vi fik var, at den ikke kunne finde en bruger at tilknytte den ordre, der blev oprettet via en foreign key. Problemet var her at da vi tilføjede “test.” præfiksen i vores SQL dokument, havde vi glemt at sætte det foran den query der henviste fremmednøglen til dens primære nøgle. Dette resulterede i at den ledte efter brugeren i public skemaet i stedet for test skemaet.

Forløbet for Java og Javascript

Vores projekt gik mere eller mindre uden komplikationer i forhold til Java og Javascript, de problemer vi stødte på havde at gøre med ting som stavefejl eller at vi havde skrevet noget forkert, som for eksempel havde vi skrevet action=”post” i stedet for method=”post” i en form i HTML, som gjorde at den ikke routede korrekt til backenden. Det største problem vi havde, havde at gøre med at en af vores “.gitignore” ikke var korrekt, så hver gang vi pushede kom de kompilerede .class filer også med, hvilket gav konflikter mellem versionerne så vi ikke kunne merge. Vi løste dette ved at køre kommandoen “git rm --cached” der fjernede de trackede filer, og derefter sørge for at vores ".gitignore" fil indeholdt “.idea/” pathen, så vi ikke trackede de kompilerede filer mere.

Forløbet for HTML og CSS

Vi startede som planlagt med at analysere det udleverede mockup og forsøgte at følge det så tæt som muligt. Efter at have opsat layoutet i HTML var teamet dog utilfredse med resultatet, da det fremstod rodet og ustruktureret.

På den baggrund valgte vi at starte forfra, men med den tidligere opsætning i baghovedet. Dette resulterede i et mere tilfredsstillende og gennemarbejdet layout, som samtidig gjorde det lettere for teamet at genbruge strukturen på tværs af hjemmesidens forskellige sider.

I forhold til vores stylesheet valgte vi at beholde størstedelen fra vores første forsøg, da stylingen generelt levede op til den standard, vi havde fastsat for projektet.

Efter at have færdiggjort hovedsiden og stylesheetet i en gennemtænkt og funktionel form, begyndte vi arbejdet med login- og registreringssiderne. Her udviklede vi indledningsvist en simpel, funktionel løsning for at sikre, at alle backend-processer fungerede korrekt. Samtidig implementerede vi grundlæggende funktioner for at verificere den samlede funktionalitet.

Da vi havde bekræftet, at alt fungerede som forventet, kunne vi nemt genanvende det etablerede layout og stylesheet. Dette sikrede, at hele hjemmesiden – og ikke kun hovedsiden – fremstod ensartet og i overensstemmelse med kundens ønsker.

Vi fortsatte denne arbejdsmetode ved først at udvikle en hurtig prototype med fokus på funktionalitet. Herefter sikrede vi, at alle funktioner fungerede korrekt, hvorefter vi tilpassede layout og design. Denne proces blev gentaget for hver side, indtil alle sider var færdiggjort, og vi stod tilbage med et gennemført og anvendeligt produkt klar til kunden.

positive og negative oplevelser under projektforløbet

Positive oplevelser

1. Online samarbejde og kommunikation

   Teamet har under projektets forløb haft meget gode erfaringer med at arbejde sammen online. Det digitale samarbejde har gjort det muligt for os at kommunikere hurtigt og effektivt, hvilket har været en stor fordel i forhold til både koordinering og fordeling af arbejdsopgaver.

   Ved hjælp af online kommunikationsværktøjer har vi løbende kunnet holde hinanden opdateret på fremskridt, udfordringer og ændringer i projektet. Dette har bidraget til en mere struktureret arbejdsproces og har mindsket risikoen for misforståelser.

   Den gode online kommunikation har desuden gjort det nemt og fleksibelt for teamet at aftale møder med kort varsel, når der opstod behov for at drøfte mere komplekse problemstillinger. I sådanne tilfælde har det været en fordel at kunne supplere skriftlig kommunikation med mundtlige samtaler, da dette har gjort det lettere at afklare idéer og træffe fælles beslutninger.

   Samlet set har det online samarbejde haft en positiv indflydelse på både arbejdsprocessen og det endelige resultat, da det har understøttet en effektiv kommunikation, god koordinering og et stærkt samarbejde i teamet.

2. Daglige møder og arbejdsstruktur

   Teamet har haft gode erfaringer med at afholde daglige møder på faste tidspunkter. Disse møder har været med til at sikre, at alle løbende blev opdateret på projektets fremdrift og aktuelle opgaver.

   Derudover har de daglige møder gjort det muligt hurtigt at afklare eventuelle udfordringer og sikre en fælles forståelse af næste skridt i processen. Samtidig har møderne bidraget til at skabe struktur i arbejdet og sikre, at alle teammedlemmer havde klare opgaver og ansvar.

   Denne tilgang har været med til at opretholde en høj arbejdsindsats i teamet, da det har skabt gennemsigtighed i arbejdsfordelingen og sikret, at alle bidrog aktivt til projektets udvikling.

3. Forståelse, fleksibilitet og arbejdsmiljø

   Selvom teamet har været gode til at holde hinanden ansvarlige, har en anden meget positiv erfaring været, at alle teammedlemmer har udvist forståelse for, at vi ikke er fagligt ens, og at vi hver især har forskellige forudsætninger.

   Derudover har der været en gensidig respekt for, at alle har personlige forpligtelser ved siden af projektet, såsom arbejde, samt individuelle udfordringer som eksempelvis sygdom. Denne forståelse har bidraget til en fleksibel og hensynsfuld tilgang til samarbejdet.

   Samlet set har dette skabt trygge og afslappede rammer i teamet, hvor der har været plads til både faglig udvikling og åben dialog, hvilket har haft en positiv indflydelse på samarbejdet og arbejdsmiljøet.

4. Feedbackkultur og løbende udvikling

   Teamets medlemmer har været meget åbne over for kritik, hvilket ofte har ført til en konstruktiv og åben dialog om, hvordan vi hver især arbejder bedst, samt hvad vi individuelt kunne forbedre eller være mere opmærksomme på fremadrettet.

   Denne dialog har omfattet alt fra faglige diskussioner omkring projektet, eksempelvis hvordan en user story skal tolkes, til mere generelle samarbejdsmæssige aspekter, såsom at blive bedre til at kommunikere med resten af teamet.

   Den åbne tilgang til feedback har bidraget til både individuel udvikling og et stærkere samarbejde i teamet gennem hele projektforløbet.

Negative oplevelser

1. Struktur Udfordringer ved online samarbejde

   Da teamet i høj grad har arbejdet online, har det i perioder medført en mindre hensigtsmæssig struktur i arbejdsprocessen. Dette har blandt andet resulteret i mange spontane møder, til tider flere gange dagligt.

   For de teammedlemmer, der samtidig har haft arbejde eller andre forpligtelser, har det gjort det vanskeligere at planlægge deres dag på en struktureret måde. Det har især haft betydning for dem, der trives bedst med en fast struktur i hverdagen.

   Som følge heraf har det i nogle tilfælde været sværere for alle teammedlemmer at deltage aktivt i arbejdsprocessen på lige fod.

2. Tidsbegrænsning og nødvendige kompromiser

   Projektets korte tidsramme på to uger har betydet, at teamet har været nødt til at indgå visse kompromiser undervejs. Dette har medført, at de opstillede krav er blevet opfyldt, men at der ikke i samme grad har været mulighed for at arbejde videre med yderligere funktionalitet eller forbedringer, hvilket er ærgerligt.

   Selvom teamet naturligvis helst havde undgået disse kompromiser, er vi samtidig bevidste om, at de har været nødvendige for at kunne gennemføre projektet inden for den givne tidsramme. Dette har derfor været en realistisk prioritering, som vi har måttet acceptere, på trods af vores ønske om at kunne levere et endnu mere gennemarbejdet produkt.

Teamets erfaringer under projektforløbet, og tanker om forbedringer

Gennem projektforløbet har teamet opnået en række værdifulde erfaringer, både i forhold til tekniske kompetencer og samarbejde. En af de væsentligste læringer har været betydningen af struktur og planlægning i et projekt med en stram tidsramme. Selvom det online samarbejde generelt fungerede effektivt og fleksibelt, oplevede vi også, at manglen på en fast struktur i perioder førte til mange spontane møder og mindre forudsigelighed i arbejdsdagen.

På baggrund af dette har vi erfaret, at det fremadrettet vil være en fordel at fastlægge en tydeligere mødestruktur og arbejdsplan fra starten af projektet. Dette kunne eksempelvis indebære faste mødetidspunkter kombineret med klare individuelle opgaver, så alle teammedlemmer bedre kan planlægge deres tid og arbejde mere selvstændigt.

Derudover har vi lært vigtigheden af prioritering under tidspres. Projektets korte varighed betød, at vi var nødt til at fokusere på at opfylde de vigtigste krav frem for at implementere ekstra funktionalitet. Fremadrettet kunne det være en fordel at arbejde endnu mere målrettet med prioritering, eksempelvis ved at opdele opgaver i “must-have” og “nice-to-have” allerede i planlægningsfasen.

En anden vigtig erfaring har været værdien af en åben feedback kultur og gensidig forståelse i teamet. Den gode kommunikation og respekt for hinandens forskellige forudsætninger har bidraget til et positivt arbejdsmiljø og et velfungerende samarbejde. Dette er noget, vi vil tage med videre og aktivt arbejde på at opretholde i fremtidige projekter.

Teknisk set har vi opnået en bedre forståelse for samspillet mellem frontend, backend og database, samt vigtigheden af korrekt fejlhåndtering og test. De udfordringer, vi mødte undervejs, har været med til at styrke vores evne til at identificere og løse problemer systematisk. Fremadrettet vil vi have fokus på at opdage og håndtere fejl tidligere i udviklingsprocessen, eksempelvis gennem mere systematisk testning undervejs.

Samlet set har projektet været en lærerig proces, hvor vi både har udviklet vores tekniske færdigheder og vores evne til at samarbejde i et team. De erfaringer, vi har gjort os, vil danne et solidt grundlag for fremtidige projekter, hvor vi i højere grad vil kunne arbejde struktureret, effektivt og målrettet.